PAL Architecture and Design

This document explains how PAL is organized and why its major interfaces and implementation choices exist. It focuses on architecture, subsystem responsibilities, trade-offs, validation strategy, and performance reasoning—not on teaching the API from the beginning. For runnable modeling workflows and public API behavior, use the API Guide. For method signatures and descriptions, use the API Reference. The Cheatsheet is the quick lookup.

Reading map

Read **Architecture and design rationale** to understand the execution layers and ownership boundaries, then **Subsystem design choices** for representation-specific trade-offs. **Testing**, **Performance**, and **Validation** explain how to verify an implementation and a scientific model. Design principles are stated where their consequences first matter rather than repeated in a separate chapter.

1. Architecture and design rationale

This chapter explains the path from a line of model code to a state change in PAL. It is intentionally more architectural than the API chapters: the goal is to explain the responsibilities of each layer, why the boundaries exist, and what those boundaries mean when debugging, extending, or benchmarking a model.

The discussion distinguishes documented contracts from engineering rationale. The public API and tests establish what users can rely on. A rationale explains the tradeoff a design serves; it does not claim an undocumented historical motivation or that no alternative design is possible.

1.1 The design problem PAL is solving

PAL targets models that repeatedly update many small pieces of state. A timestep might inspect thousands of neighborhoods, move individual agents, update a population count at every site, solve a field equation, or combine several of these operations. Writing every inner operation as a Python call is expressive but can impose substantial interpreter overhead. Writing the whole model in C could reduce that overhead, but would make iteration and experimentation less convenient for Python users.

PAL's architecture splits the problem by frequency and responsibility. Python remains the control language for constructing the model, selecting parameters, loading inputs, managing experiments, and deciding when to draw or checkpoint. Repeated computation is expressed in functions that PAL can prepare for Numba compilation. The native core supplies low-level state and kernels behind that interface.

This split does not mean that all Python code is slow or that every model function should be compiled. It puts optimization effort where repeated work makes it worthwhile. A function called once during setup usually needs less attention than a loop executed at every site over millions of timesteps. Start with clear model code, identify the repeated work, and benchmark before moving more logic into compiled kernels.

1.2 The layers and their responsibilities

A useful mental model is to separate PAL into several cooperating layers:

1. **Public API and Protocol declarations.** These define the names, signatures, type-facing interface, and expected user-visible operations. They support readable calls, type annotations, and autocomplete.

2. **Python constructors and wrappers.** These create objects, translate public arguments into implementation calls, and expose native-backed state through a Python-friendly interface.
3. **Numba-compatible wrappers and overloads.** These let compiled model functions call PAL operations while preserving supported argument forms and return behavior.
4. **The `pal.njit` preparation step.** This transforms selected PAL-specific constructs before compilation, including neighborhood/box iteration and some source-aware diagnostics.
5. **The native C core.** This provides low-level storage and computational routines used by the Python/Numba-facing layer.
6. **Tests, examples, and generated documentation.** These act as executable and written statements of the public contract, checking that the layers agree.

These are responsibilities, not necessarily six separate packages; several may share a source file. Keeping them conceptually distinct helps localize defects. A constructor bug, a Numba typing error, a native-state problem, and a stale API declaration require different fixes.

1.3 The native core and Python interface

The native core gives PAL a place to implement low-level operations without requiring users to write C for ordinary model logic. Native-backed objects can hold state in a representation suited to repeated updates, while the Python-facing API offers recognizable constructors, indexing, and methods.

The wrapper is important because the native implementation is not itself the public contract. A C function may accept a particular pointer or integer layout internally, but the Python API should expose a stable and understandable operation. The wrapper layer is where public arguments, supported types, safe-mode validation, and implementation details meet.

This separation creates a maintenance obligation: behavior must remain coherent across the wrapper and native layers. If the Python method accepts a keyword argument but its Numba overload silently binds that keyword differently, the layers disagree. If a native routine handles a one-dimensional coordinate differently from the safe wrapper, a mode-specific defect can result. PAL's parity tests therefore compare outcomes across paths rather than assuming that one layer automatically inherits the correctness of another.

1.4 Native-backed state and object lifetime

A PAL object is more than a Python container. Its public methods operate on state managed through PAL's native-backed implementation. The Python object is the user's handle to that state; it is not safe to assume that every internal value is an ordinary Python list or NumPy array that can be edited freely.

This matters for lifetime and mutation. Operations such as creating, moving, or disposing an agent must update the grid's internal structures consistently. Writing directly into an internal count or pointer-like representation could bypass the rules that keep occupancy and agent handles aligned. PAL exposes explicit methods for state changes so the implementation can preserve those invariants.

The same distinction explains why a returned array or slice must be interpreted according to its documented contract. A detached copy is a snapshot, not a live view into native memory. Editing it changes the copy until the result is explicitly assigned back. This is less magical than pretending every slice is a mutable view and makes ownership clearer at the Python boundary.

Native-backed objects can support pickling, but pickling an object is not equivalent to serializing the entire simulation process. A model's complete checkpoint may also need parameters, timestep, other coupled objects, external state, and random-number state. See the API Guide's saving section for the checkpoint contract and its limitations.

1.5 The public API is a contract, not a mirror of the C implementation

PAL's public names and signatures are designed around modeler-facing concepts: a grid site, an agent handle, a population count, a neighborhood, a field, or a random draw. The native core may use lower-level representations that do not resemble those concepts directly. That is acceptable as long as the public behavior remains consistent.

Protocol declarations are part of this contract because users and tools read them for annotations and autocomplete. But a declaration is not executable proof. Likewise, a docstring or manual paragraph cannot make unsupported runtime behavior valid. The actual implementation, declarations, tests, and generated references need to describe the same public surface.

This is why API changes should be made as a coordinated change. When altering an argument name, return type, indexing rule, or safe-mode behavior, check the Python method, the Numba overload, relevant Protocol declaration, documentation generator, examples, and regression tests. A change that works in one path but breaks another is not complete merely because the primary example still runs.

1.6 Python and Numba execution paths

Python execution is useful for interactive exploration, setup code, inspection, and small operations. Numba-compiled execution is useful for repeated loops where Python dispatch would be costly. PAL supports both because a scientific workflow naturally mixes these kinds of work.

Supporting both is not free. Numba imposes typing and compilation constraints that ordinary Python does not. Native-backed objects need compatible wrappers or overloads; argument types and names must be represented in forms Numba understands; and code transformations must produce valid compilable functions. An operation can therefore work in Python and fail at compile time, or compile successfully but behave differently in an edge case.

PAL treats parity as a correctness requirement for supported usage, not as a promise that Python and compiled code have identical internals or performance. Compare observable behavior: return values, mutations, errors where they are part of the contract, boundary handling, and state after the operation. If a difference appears, reduce it to a minimal reproducer and test both paths.

1.7 Model-code transformation with `pal.njit`

PAL's `njit` interface is not simply a spelling alias for Numba's decorator. Before compilation, it can prepare functions that use PAL-specific syntax and diagnostics. In particular, neighborhood and Box iteration are model-friendly abstractions that may need to be expanded into forms that the compiler can type and execute efficiently.

The transformation lets users express spatial operations directly instead of writing coordinate conversions in every loop. It also gives PAL a place to add source-aware safe-mode diagnostics when argument annotations provide enough type information.

Transformations add a layer that users should understand when debugging. The Python source a modeler reads is not necessarily the exact function body Numba receives. A traceback or typing error may refer to transformed code or a synthetic compilation context. Annotations improve the information available to the transformer, but they do not guarantee that every error can be attributed to a perfect source location.

When a failure occurs only under `@pal.njit`, do not immediately rewrite the scientific algorithm. First isolate the smallest function that reproduces it, confirm argument annotations and supported API usage, and compare it with a Python version. Then determine whether the problem lies in the transformation, Numba typing, a wrapper, or the underlying operation.

1.8 Relative-offset neighborhoods

A neighborhood such as Moore or Von Neumann is naturally a pattern around a location. PAL represents it as relative integer offsets, then applies it to a specific grid position with `grid.Hood(...)`. This separates the reusable pattern from the grid location where it is used.

The distinction also makes boundary behavior explicit. Applying a hood must account for grid dimensions and wrapping flags; the relative pattern alone does not know whether a particular neighbor falls outside a nonperiodic domain. PAL's application step can translate offsets into coordinates, wrap periodic axes, and omit out-of-domain sites according to the public contract.

This avoids storing a separate absolute-neighbor list for every cell and lets the compiled path transform high-level neighborhood loops. It does not guarantee unique destinations: small or wrapped domains can map multiple offsets to the same site. Whether to count those repeated contributions or deduplicate them depends on the model, so choose and test the intended behavior.

1.9 Separate update and commit operations

PAL makes a distinction between changes that are being accumulated and changes that have become the current state. For `PopGrid` and `PDEgrid`, `Add` queues a delta and `Update` commits pending changes. This makes the timestep boundary visible in code.

This distinction matters mathematically. If every new site value must be computed from the same old field, immediate mutation can make results depend on iteration order. Queued deltas let the model calculate changes before committing them. Direct indexing remains available when immediate assignment is intended; PAL permits order-dependent algorithms but makes the choice explicit.

Agent updates have different semantics. `All()` snapshots which handles will be visited, but it does not snapshot the world. If one agent reproduces, later agents may see the newborn's occupancy even though the newborn is not part of the original handle snapshot. This is a deliberate and useful pattern for sequential agent models, but it is not equivalent to a synchronous cellular update.

PAL does not impose one scheduler because these update schemes represent different models. Choose the intended timing and test it with a small case whose result can be calculated by hand.

1.10 Process-wide safe and fast modes

Safe mode performs selected validation and provides diagnostics that are useful while developing a model. Fast mode removes selected checks from performance-sensitive paths. Keeping one mode active for a process makes the execution contract predictable for objects created within that process; PAL locks the choice at first construction.

A process-wide mode also avoids pretending that an individual object can be switched independently when calls across objects may share compiled functions or native machinery. The consequence is operational: call `pal.FastMode()` before creating any PAL object or window, and run safe and fast test suites in separate processes.

Fast mode should be treated as a measured optimization, not a correctness mode. It can reduce validation overhead; it cannot repair invalid model assumptions, unsafe numerical parameters, or a wrong update order. Safe mode can catch selected contract violations, but it cannot establish that the biological or physical equations are appropriate.

1.11 Separating visualization from simulation

PAL separates simulation state from rendering. The model changes its grids and agents; the drawing layer turns the current state into pixels or geometric primitives. This permits the same model to run headlessly, draw every timestep, draw intermittently, or save output without changing the scientific update rules.

The split is important for performance. Rendering can be expensive and can obscure the cost of the simulation itself. When benchmarking, measure the update kernel separately from drawing, then measure end-to-end runtime if that is what users experience. When validating a model, avoid treating a plausible-looking image as proof of correct state transitions.

The two main window abstractions serve different needs. A pixel window is useful when state maps directly to a pixel lattice; an OpenGL window supports geometric 2D/3D drawing. Headless modes enable automated image/output workflows where supported, but they do not test every interactive display behavior or guarantee that every graphics backend is available on every machine.

1.12 Random-number state and reproducibility

PAL's random functions expose an explicit stream that model code can seed and consume from Python or compiled functions. This supports repeatable experiments when the initial state, parameters, seed, and order of random calls are held fixed.

The random-call order is part of reproducibility. Adding a draw to one branch changes the state of the stream and may change all subsequent random outcomes. Consequently, two versions of an algorithm can use the same seed but produce different trajectories because they consume random values differently. This is expected, not evidence that seeding failed.

Randomness is also separate from model-object state. Saving a grid or agent structure does not automatically preserve every random generator used by the process. Exact stochastic continuation requires explicit treatment of PAL's stream and any external RNG state. Record seeds and configurations for repeatable experiments, and do not promise bit-for-bit continuation from an object checkpoint alone.

1.13 Layered testing

No single test style can establish every kind of correctness. PAL uses several complementary forms of evidence:

- **Focused API tests** check individual public operations, argument forms, boundary cases, and error behavior.
- **Python/Numba parity tests** check that supported calls agree across execution paths.
- **Safe/fast tests** check behavior under both validation settings, which must be initialized in separate processes.
- **Mathematical invariants** check properties such as conservation, coordinate identities, nonnegativity, or known gradients.
- **Integrated examples** exercise multiple components together under realistic model control flow.
- **Build and CI checks** rebuild the native core and run the suite against the repository source, reducing the risk of accidentally testing a stale binary or incomplete local setup.

Each layer catches a different class of defect. A focused test localizes a regression; a parity test catches path divergence; an invariant can catch an error shared by multiple implementations; and an integrated model can expose interaction bugs that isolated unit tests miss. Passing these tests still does not prove that a scientific hypothesis is correct, so model validation remains a separate responsibility (see the validation chapter below).

1.14 Reliable performance measurement

PAL includes compilation, native calls, validation, model logic, and sometimes rendering in one user workflow. A timing that mixes these costs can be misleading. The first call to a compiled function may include compilation and specialization; later calls measure steady-state execution. A benchmark that measures only the first call may primarily be measuring compilation.

For useful comparisons, separate cold compilation from hot runtime, use the same workload and state, repeat measurements, and check outputs for equivalence. Test the safe and fast modes in separate processes. Include the dimensions, population size, neighborhood size, number of timesteps, and whether rendering is enabled. A microbenchmark can reveal a local cost, but the actual model workload decides whether that cost matters.

The design goal is not to maximize every isolated benchmark at the expense of a comprehensible API. A performance change should be accepted when it is reproducible, preserves behavior, and improves a representative workload. Prefer algorithmic reductions in unnecessary work before complicating code for a tiny or noisy gain.

1.15 How to trace a PAL operation when debugging

When an operation behaves unexpectedly, follow the layers in order:

1. **Check the public contract.** Confirm the method, argument forms, coordinate convention, and return type in the API Guide and Reference.
2. **Reduce the model.** Build the smallest example with a tiny grid and a known expected result.
3. **Compare Python and compiled paths.** If the error appears only under `@pal.njit`, inspect annotations and transformation-related constraints.
4. **Compare safe and fast modes in separate processes.** A safe-mode exception may reveal a contract violation that fast mode does not check.
5. **Inspect the state transition.** Verify whether a method mutates immediately or queues a change for `Update()`; verify whether iteration uses a snapshot or current occupancy.
6. **Check geometry and boundaries.** Confirm dimensions, wrapping, index conversions, and neighborhood duplicates.
7. **Check independent invariants.** Test a conservation law, exact coordinate identity, or hand-computed update.
8. **Only then localize the implementation layer.** Determine whether the defect belongs to the public wrapper, Numba overload/transformer, native core, or visualization layer.

This sequence moves from observable contract to minimal evidence before diving into implementation details. It reduces the risk of "fixing" a wrapper symptom by changing the model's scientific behavior.

1.16 A concrete execution trace

Consider a model step that reads a field value, changes an agent, and queues a population delta. At the **model layer**, the author chooses the order of those operations. The **public PAL API** resolves coordinates, handles, and arguments. In ordinary Python execution, Python-facing wrappers perform the supported operations; in a compiled step, PAL's preparation and Numba-compatible call path must preserve their observable meaning. Both paths ultimately work with the relevant native-backed state. A later `Update()` on the population grid commits the queued delta; it is not implied by a field read or an agent movement.

This trace exposes three independent questions during debugging: **Was the right site or handle selected? Was the right operation dispatched? Was the state committed at the intended point?** A failure in any one can produce a scientifically wrong timestep despite apparently reasonable model code.

The execution path is conceptual rather than a promise that every call crosses every layer. Some operations are resolved in wrappers or compiled code; consult the actual implementation when profiling call overhead or changing a kernel.

1.17 Maintaining state and ownership boundaries

A PAL object's public lifetime is managed through its Python-facing object, while native-backed state may have its own allocation and representation. The important contract is not a presumed pointer layout: it is that supported operations preserve the object's invariants and that serialized state is reconstructed without relying on process-local addresses.

For a new operation, identify which layer owns each invariant before editing it. An agent move affects both position and occupancy. A disposal affects liveness and future handle validity. A population update must reconcile queued deltas with current counts and any capacity constraints. A PDE transport operation must respect the field's configured geometry, boundary treatment, and commit semantics. Bypassing the public mutation path may leave those related pieces inconsistent.

Do not expose a mutable native buffer as a convenience API without defining its ownership, lifetime, invalidation, and synchronization rules. In particular, a detached NumPy copy and a live native-backed view have different mutation contracts.

1.18 How to extend a public operation

1. Define the intended model-facing behavior, including dimensions, positional and keyword arguments, return type, mutation timing, and failure cases.
2. Identify the owning implementation and any native state invariants; decide whether the operation belongs in an existing wrapper or requires a native kernel.
3. Make the supported Python and `@pal.njit` paths agree. If neighborhood or Box syntax is involved, inspect PAL's transformation rules rather than assuming Numba can compile an arbitrary Python iterator.
4. Add a hand-checkable regression case, including boundaries and invalid inputs where safe-mode validation applies. Compare Python and compiled results, and run safe and fast tests in separate processes.
5. Update public Protocol declarations and source docstrings, regenerate the API Reference, and update the API Guide if user-visible behavior changed.
6. Benchmark cold compilation and steady-state runtime only when performance is part of the change.

This order prevents documentation from becoming an unsupported promise and keeps low-level implementation choices subordinate to the user-visible contract.

2. Testing PAL

PAL's `Testing/` suite exercises public behavior across geometry and wrapping, Grid/IList/Multinomial/RNG operations, AgentGrid and PopGrid state changes, PDE transport and boundaries, compiled Python/Numba parity, visualization, integrated models, and historical regressions. Many tests use independent invariants or small reference calculations rather than reproducing PAL's implementation.

Run the full suite from the repository root in separate safe and fast processes:

```
PAL_TEST_MODE=safe python -m pytest -q Testing
PAL_TEST_MODE=fast python -m pytest -q Testing
```

PAL locks its process-wide mode at first construction, so safe and fast should not be combined in one Python process. Fast mode intentionally skips some validation tests. The CI workflow runs both modes on Linux, verifies all generated documentation artifacts are current, runs the API benchmark after successful tests, and separately runs representative examples in safe and fast modes on Windows. Headless OpenGL tests require the relevant graphics backend.

When adding or changing model code, start with the smallest test that expresses the intended public behavior, then run the affected test file before the full suite. Prefer conservation laws, coordinate identities, exact small examples, and Python/compiled agreement over tests that duplicate PAL's internal formulas.

3. Subsystem design choices

The governing design choices are: use distinct representations for distinct state semantics; make coordinate systems and update timing explicit; expose one public contract through Python and compiled paths; and prefer native storage for repeated work without requiring model authors to program in C. The following sections describe how those choices affect each subsystem.

The following sections examine subsystem-specific contracts and the implementation tradeoffs behind them. For complete call signatures and usage examples, see the API Reference and API Guide.

3.1 Grid: typed site state and explicit indexing

A `Grid` represents one typed value per lattice site. The element type is chosen at construction, which makes the intended state representation explicit and allows the implementation to use compact native storage rather than a Python object at every site. Use a type that matches the model's meaning: integer labels for categories, Boolean values for masks, and floating-point values for continuous attributes.

Indexing supports both linear site indices and coordinate forms, but these are different views of the same lattice. Use coordinate indexing when the equation is naturally spatial; use a linear index when iterating sites or passing a site identifier between APIs. In more than one dimension, coordinate tuples are often easier to review because the axes remain visible. The conversions `ToI` and `ItoX` / `ItoY` / `ItoZ` are explicit so the mapping can be tested rather than guessed.

Grid slices are detached NumPy copies, not live native-backed views. This avoids pretending that a slice is a mutable view into storage when it is actually materialized for Python. Read a slice as a snapshot, then assign the result back if changes should affect the Grid. In safe mode, assignment validates relevant type, range, and nonfinite conditions; do not rely on fast mode to diagnose invalid values.

3.2 AgentGrid: handles, occupancy, and mutation

An agent handle is distinct from its position. The handle identifies the living agent; position determines where it is located. This separation lets a model retain identity while an agent moves, and lets queries report handles without copying whole Python objects. The model remains responsible for the meaning and lifecycle of any additional per-agent state.

Stackability is an explicit constructor choice because allowing multiple agents at a site changes the model's state space and affects occupancy logic. A nonstackable grid rejects a second occupant in safe mode. Do not enable stackability merely to suppress an error: decide whether multiple occupancy is biologically or physically meaningful.

The distinction between `All()` and a live iteration matters for mutation. `All()` returns a snapshot of handles and is therefore the appropriate pattern when the loop can create or dispose agents. A snapshot freezes membership, not agent state or grid occupancy. Each later query observes current occupancy. If newborns must not act until the next timestep, build the snapshot before processing the timestep; if all agents must compute from the same old world, snapshot the required state as well.

The `counts` property is a read-only occupancy query view, not a writable alternate path to agent state. Changing a count directly would bypass agent handles, placement rules, and lifecycle bookkeeping. Query occupancy through this view; create, move, and dispose agents through the AgentGrid API.

3.3 PopGrid: counts instead of individual objects

A `PopGrid` stores integer population counts per site. It is intended for populations whose members are interchangeable for the questions being asked. That can greatly reduce storage and update overhead compared with representing every individual as an agent, but it removes individual-level history and identity.

Its `Add` operation queues a delta; `Update` applies pending changes; `Reset` clears current state and pending deltas. The explicit delta buffer makes it possible to compute a set of changes before committing them. For a movement event from site A to site B, queue a negative delta at A and a positive delta at B, then update once. If the model reads current counts during this phase, it sees the pre-commit counts, which helps make the update independent of visitation order.

The `capacity` option is a per-site cap. It is not the total population limit. `All()` returns the indices of nonzero sites, not the counts themselves; read the count at each returned index when the value is needed. Keeping these semantics distinct avoids confusing the set of occupied sites with the population data stored there.

3.4 PDEgrid: numerical operations with explicit assumptions

A `PDEgrid` stores a lattice scalar field. Its Cartesian, interface-based, masked, ADI, and radial transport methods address different geometries, coefficient representations, boundaries, and numerical schemes. The method is therefore part of the mathematical model, not merely a performance setting.

Call `SetTimeSpaceStep` to specify the time and spatial scales used by transport methods. These values are part of the discretized equation. A timestep that is too large can produce unstable or inaccurate results even if every API call is valid. Boundary conditions likewise belong to the mathematical problem: fixed-value, face-dependent, and periodic boundaries can produce different solutions.

Transport routines accumulate changes for `Update()` to commit. That separation does not guarantee stability, positivity, or conservation. Validate against a known field, check boundary behavior, and test convergence as timestep and grid spacing change.

3.5 IList: reusable collections for compiled workflows

`IList` stores reusable integer collections in Python and compiled code. For example, a model can collect empty neighboring sites and choose one for reproduction without allocating a new collection at every step.

`All()` returns a detached copy; `Iter()` traverses the current list without copying. Use the former when results must survive later mutations and the latter for one-pass traversal. Clear reused lists before repopulating them.

3.6 Randomness: one explicit PAL stream

`Seed`, `Random`, and `RandInt` share PAL's random stream across Python and compiled code. Each draw advances the stream, so even one additional draw changes subsequent outcomes. Reproducibility requires matching the seed, initial state, parameters, and draw order.

Use PAL's RNG functions when a model should share one reproducible stream across PAL calls. External RNGs have independent state and need their own seed or checkpoint handling. A PAL object checkpoint does not by itself serialize the random stream, so exact stochastic continuation requires separately preserving or reconstructing random state. Do not claim that a seed alone guarantees identical outcomes after changing the algorithm.

3.7 Multinomial sampling: configuration versus sample state

`NewMultinomial(other)` copies solver configuration, not the source sampler's current multinomial remainder. The new sampler therefore starts with independent sampling state. Both samplers must use the same process-wide safety mode.

3.8 Visualization: simulation state versus presentation

PAL separates simulation state from rendering. `PixWindow` exposes a writable pixel buffer for lattice-style images; `OpenGLWindow` draws 2D/3D geometry. They are distinct rendering interfaces.

Write pixel colors or OpenGL primitives, then call the window's `Update()` to publish a frame. Rendering every timestep helps inspection but may dominate runtime; long simulations can render selected steps. Keep drawing separate from the computational

kernel unless using an explicitly supported compiled drawing API.

Headless rendering exists so output can be generated and tested without a visible interactive window, but it still depends on the relevant image or graphics backend. A headless test validates the supported off-screen path; it cannot fully validate window placement, user input, or a display server's lifecycle.

3.9 Source-aware diagnostics and AST transformation

PAL's `njit` wrapper transforms selected constructs, including neighborhood and Box iteration, before Numba compilation. Type annotations on PAL arguments also support source-aware safe-mode diagnostics.

Python syntax alone does not guarantee matching compiled behavior: AST rewriting, wrappers, native code, and Numba typing all contribute. When a problem appears only under compilation, reduce it to a minimal function and compare Python and compiled execution.

Source-line diagnostics depend on sufficient type information, including annotations for relevant PAL arguments. Unannotated arguments do not promise the same diagnostics. Treat annotations as part of writing clear compiled model code, not as a guarantee that every error can be attributed to an exact source expression.

3.10 Public API and implementation consistency

Python and `@pal.njit` use different wrappers and diagnostics, but must agree on supported inputs and public results. Python validation and Numba overloads are implementation details, not separate user-facing contracts.

This is why the project uses API coverage inventories, Python/compiled parity tests, integrated examples, and safe/fast test runs. A type declaration or docstring is not sufficient evidence if runtime behavior disagrees. Conversely, a one-off implementation detail should not be elevated to a public promise unless the tests and documentation support it. When changing a contract, update the implementation, declarations, generated reference, manual, examples, and tests as one coherent change.

4. Performance: measure the right thing

For maintainers, a change is complete only when its observable behavior agrees in Python and compiled code, in safe and fast modes where applicable, and in the public declarations, tests, and generated documentation. A practical extension sequence is: define the user-visible contract; implement the native and wrapper behavior; update Numba overloads or transformations if needed; add focused and parity tests; update Protocol declarations and docstrings; regenerate documentation; then benchmark representative workloads. Do not infer implementation details from this high-level sequence: inspect the current source and tests before changing ownership or compilation machinery.

4.1 Cold compilation is not steady-state runtime

The first call to a compiled function may include Numba compilation and specialization. Later calls can be much faster, particularly when caching is enabled. Timing one first call and reporting it as the simulation's runtime mixes two different costs.

For a useful benchmark, measure cold compilation in a fresh process and steady-state runtime over repeated timesteps separately. Use the same model state, workload, seed, and operation count for each comparison. Report medians over repeated runs rather than relying on one timing. If comparing safe and fast modes, use separate processes because the mode is fixed at construction.

4.2 Optimize the actual workload

A faster microbenchmark does not guarantee a faster full model. An optimization may help a large agent sweep but hurt a small grid, or reduce steady-state time while increasing compilation cost. Before changing implementation, define the workload that matters: number of sites or agents, number of timesteps, neighborhood size, mutation rate, transport method, and mode.

Use a before/after benchmark with fresh-process cold compilation and repeated hot runtime. Keep workload sizes identical, check outputs for equivalence, and investigate any reproducible regression. Treat tiny differences as noise until repeated measurements show otherwise. Prefer simpler correct code when a performance gain is not robust.

4.3 Know what fast mode cannot fix

Fast mode can reduce selected validation overhead; it cannot eliminate the fundamental cost of updating millions of state elements, choosing an inefficient representation, doing unnecessary neighborhood work, or solving an unstable numerical scheme. Before optimizing low-level code, ask whether the model's algorithm is doing unnecessary work. Replacing individually represented interchangeable agents with counts, reusing a neighborhood list, reducing repeated conversions, or drawing less often can have a larger impact than a local kernel tweak.

Do not optimize by removing a check merely because it appears in a profile. First establish that the check is responsible for a meaningful fraction of runtime, that the faster behavior is safe for the intended workload, and that tests cover the affected contract.

5. Validation: prove the model, not just the program

5.1 Start with a hand-calculable case

Before running a large simulation, reduce the domain and timestep count until the expected answer can be computed manually. A two-site transfer can establish whether queued population changes conserve the total. A linear field can establish whether indexing and finite differences produce the expected gradient. A tiny periodic grid can show exactly which neighbor lies across a boundary.

Small examples are powerful because they turn vague visual plausibility into an assertion with a known answer. Keep such examples as regression tests after the model grows.

5.2 Test invariants and edge cases

Choose invariants from the scientific model. For a closed diffusion problem, total mass should be conserved within a stated numerical tolerance. For an internal population transfer, the total should remain constant and counts should stay nonnegative. For an agent model, every live handle should refer to a living agent and occupancy should obey the stacking rule. For geometry, conversions should round-trip and wrapping should return the expected coordinates.

Test the cases most likely to break assumptions: the first and last site, wrapped and nonwrapped axes, empty neighborhoods, a single agent, zero or maximal allowed counts, and updates that add and remove state in the same timestep. A model that works only away from boundaries has not yet validated its geometry.

5.3 Separate API correctness from scientific validity

Safe-mode checks and passing unit tests show that operations respect tested contracts. They do not establish that the chosen growth law, diffusion coefficient, update schedule, stochastic distribution, or boundary condition is scientifically justified. Validate the numerical method against known solutions where possible; test convergence as grid spacing and timestep change; and compare stochastic results across independent seeds.

Document assumptions alongside the model: whether agents update sequentially or synchronously, whether boundaries are reflecting, fixed, or periodic, whether rates are per unit time or per step, and whether counts or fields represent densities or totals. These assumptions determine what the simulation means.

6. Using the rest of the documentation

Use the API Guide for the first model, worked workflows, and public API behavior. Use the API Reference for exact signatures and method descriptions, and the Cheatsheet for common patterns. The `Examples/` directory contains complete models. The Testing guide describes regression coverage and how to run the test suite.